LAPORAN TUGAS KELOMPOK

Mata Kuliah Sistem Operasi (USA-WP2260110)

Implementasi Komunikasi Antarproses (IPC)

Kode dan judul tugas	K-1 · Implementasi Komunikasi Antarproses (IPC)
Kelas / tema	Cloud dan Layanan Digital UMKM
Nama kelompok	Kelompok 8
Nama host VM	srv-toko-250660121024
Minggu / tanggal kumpul	Diberikan minggu ke-4, dikumpulkan sebelum pertemuan minggu ke-5
Kepingan puzzle	Kepingan #4 Level 2 Puzzle UTS
Dosen pengampu	Muhammad Agreindra Helmiawan, S.Kom., M.T

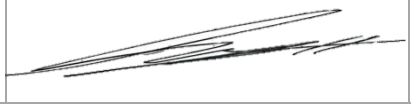
Anggota kelompok

No	Nama	NIM	Peran
1	Diza Asmiranda Subianto	250660121024	Leader
2	Naya Naira Zahra	250660121067	Anggota 1
3	Muhammad Rizky Zamzami	250660121044	Anggota 2

Program Studi S1 Informatika
Fakultas Teknologi Informasi · Universitas Sebelas April
2026

Pernyataan Keaslian

Kami yang bertanda tangan di bawah ini menyatakan bahwa laporan ini merupakan hasil kerja kelompok kami sendiri. Seluruh sumber yang dikutip telah dicantumkan, penggunaan AI generatif telah diungkapkan pada Lampiran, dan setiap anggota berkontribusi sesuai lembar kontribusi.

Nama / NIM	Tanda tangan	Tanggal
Diza Asmiranda Subianto 250660121024		
Naya Naira Zahra 250660121067		
Muhammad Rizky Zamzami 250660121044		

Daftar Isi

Pernyataan Keaslian 2
1. Pendahuluan 4
1.1 Latar belakang kasus 4
1.2 Tujuan tugas 4
1.3 Ruang lingkup 4
2. Pembagian Peran dan Rencana Kerja 5
3. Dasar Teori Singkat 5
3.1 Inter-Process Communication (IPC) 5
3.2 Remote Procedure Call (RPC) 6
3.3 Socket 6
3.4 Message Passing 6
3.5 Order Service dan Stock Service 7
4. Lingkungan Praktik 7
5. Langkah Pengerjaan 7
5.1 Langkah 1: Verifikasi Kesiapan Lingkungan VM 7
5.2 Langkah 2: Implementasi Stock Service (RPC Server) 8
5.3 Langkah 3: Implementasi Order Service (RPC Server) 10
5.4 Langkah 4: Pengujian Komunikasi RPC 10
5.5 Langkah 5: Pembuatan Layanan Alternatif Berbasis Socket TCP 11
6. Hasil dan Analisis 13
6.1 Hasil 13
6.2 Pengujian 13
6.3 Analisis 13
7. Kesimpulan 14
8. Kepingan Puzzle 14
Daftar Pustaka 15
Lampiran A. Kode Program dan Log 16
Lampiran B. Lembar Kontribusi Anggota 21
Lampiran C. Log Kerja Kelompok 22
Lampiran D. Pernyataan Penggunaan AI Generatif 23
Lampiran E. Daftar Periksa Sebelum Mengumpulkan 23
Swa-nilai berdasarkan kriteria lembar tugas 23

1. Pendahuluan

1.1 Latar belakang kasus

Perkembangan teknologi informasi memungkinkan berbagai layanan dalam sebuah sistem berjalan secara terpisah, tetapi tetap dapat saling berkomunikasi, contohnya adalah sistem toko online yang mempunyai layanan untuk mengelola pesanan dan layanan untuk memeriksa ketersediaan stok produk.

Pada sistem toko online layanan pesanan perlu mengetahui apakah stok produk mencukupi sebelum pesanan dapat diproses. Apabila kedua layanan berjalan sebagai proses yang terpisah, diperlukan mekanisme komunikasi antarproses atau *Inter-Process Communication* (IPC) agar keduanya dapat bertukar informasi.

Pada tugas kelompok Sistem Operasi kami mengimplementasikan komunikasi antara Order Service dan Stock Service menggunakan Remote Procedure Call (RPC) serta socket. Implementasi ini digunakan untuk mempelajari proses pertukaran pesan, pengiriman permintaan, penerimaan respons, serta kelebihan dan keterbatasan mekanisme komunikasi yang digunakan.

1.2 Tujuan tugas

Sub-CPMK-4; CPMK061, CPMK063, CPMK081, CPMK102, CPMK11

Tujuan pengerjaan tugas ini adalah:

- Memahami konsep komunikasi antarproses dalam sistem operasi.
- Mengimplementasikan komunikasi antara Order Service dan Stock Service menggunakan RPC dan socket.
- Memahami mekanisme *message passing* dalam pertukaran informasi antarlayanan.
- Menguji komunikasi dan respons layanan terhadap permintaan pemeriksaan stok.
- Menganalisis kelebihan, keterbatasan, dan kemungkinan pengembangan implementasi IPC.
- Mendokumentasikan hasil implementasi, pengujian, dan pembagian kontribusi anggota kelompok

1.3 Ruang lingkup

Ruang lingkup pengerjaan tugas ini meliputi:

1. Penggunaan lingkungan Ubuntu Server yang disiapkan melalui modul praktikum.
2. Pembuatan atau penyesuaian Order Service sebagai pihak yang mengirim permintaan.
3. Pembuatan atau penyesuaian Stock Service sebagai pihak yang memeriksa ketersediaan stok.
4. Implementasi komunikasi menggunakan XML-RPC dan socket.
5. Pengujian permintaan stok yang mencukupi, stok tidak mencukupi, serta kondisi masukan yang tidak valid.

6. Analisis komunikasi antarlayanan berdasarkan hasil pengujian.

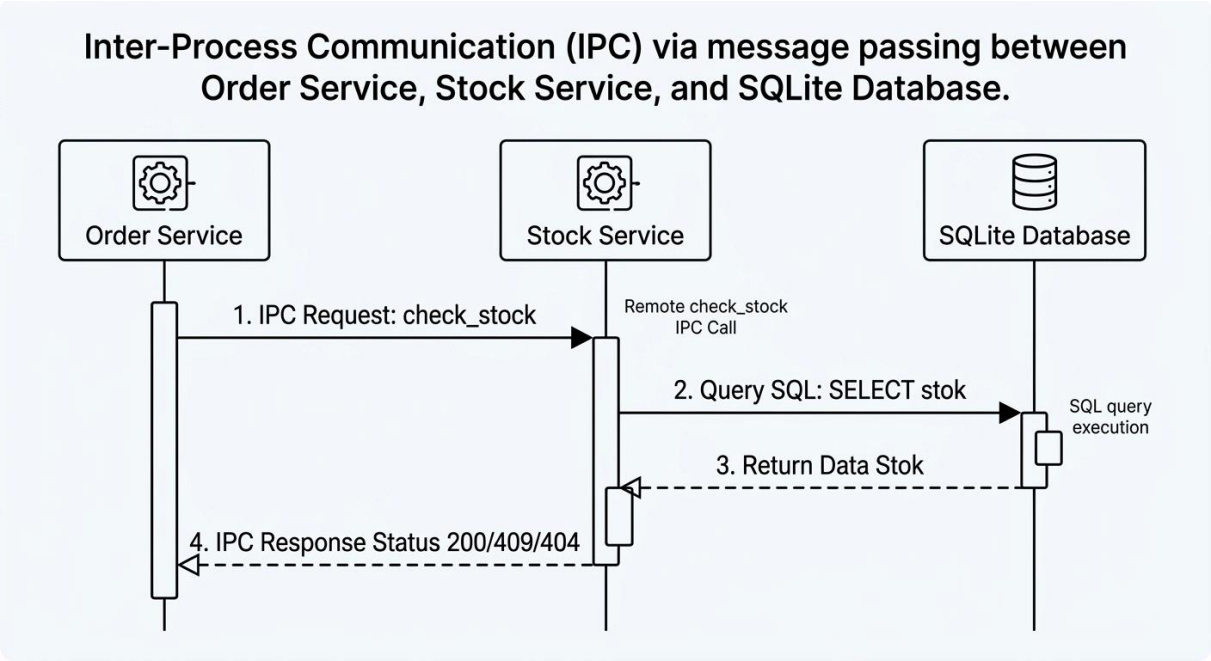
2. Pembagian Peran dan Rencana Kerja

Anggota	Peran	Tanggung jawab	Tenggat
Diza Asmiranda Subianto	Penguji	Merancang arsitektur kode IPC (XML-RPC & TCP Socket), implementasi skrip utama server-client, serta pengujian sistem di VM.	09 Oktober dan 13 Oktober 2026
Naya Naira Zahra	Membantu pengujian dan penyusunan laporan	Membantu penulisan skrip server RPC dan membantu perbaikan dan melegkapi laporan	09 Oktober dan 13 Oktober 2026
Muhammad Rizky Zamzami	Penyusun laporan	Menyusun draf laporan lengkap dari Bab 1 sampai Bab 5, perapian format dokumen.	08 Oktober dan 12 Oktober

3. Dasar Teori Singkat

3.1 Inter-Process Communication (IPC)

Inter-Process Communication (IPC) merupakan mekanisme yang memungkinkan proses untuk bertukar pesan atau data. IPC dapat dilakukan melalui mekanisme seperti pipes, message queues, atau shared memory.(Herdiana et al., 2025) IPC diperlukan ketika suatu pekerjaan dibagi ke dalam beberapa proses yang perlu berkomunikasi untuk menyelesaikan tugas tertentu. Pada kasus toko online, Order Service dan Stock Service dapat berjalan sebagai proses terpisah. Order Service membutuhkan informasi dari Stock Service sebelum menentukan apakah suatu pesanan dapat dilanjutkan.



Gambar 3.1 *Diagram Alur Komunikasi Inter-Process Communication (IPC) antara Order Service dan Stock Service.*

3.2 Remote Procedure Call (RPC)

3.3 Socket

Socket merupakan antarmuka komunikasi yang memungkinkan dua proses bertukar data melalui koneksi jaringan. Sedangkan menurut (Kelana et al., 2017) Pemrograman socket adalah cara untuk menggunakan komponen/API (Application Programming Interface) socket untuk membuat sebuah aplikasi jaringan yang bekerja dengan sistem Client-Server karena dengan socket memungkinkan kita untuk melakukan suatu komunikasi data. Salah satu jenis socket yang umum digunakan adalah TCP socket, yang menyediakan komunikasi berbasis koneksi dan pengiriman data secara berurutan serta andal selama koneksi berjalan dengan baik.

Pada implementasi ini, socket dapat digunakan agar Order Service mengirim pesan pemeriksaan stok kepada Stock Service. Stock Service membaca pesan, memproses permintaan, kemudian mengirimkan respons kembali kepada Order Service.

3.4 Message Passing

Message passing adalah mekanisme IPC dengan cara satu proses mengirim pesan kepada proses lain. Penerima memproses pesan tersebut dan dapat mengirimkan pesan balasan.

RPC dan socket pada tugas ini sama-sama digunakan untuk pertukaran pesan antarlayanan. Perbedaannya terletak pada tingkat abstraksi: RPC menyediakan pemanggilan fungsi melalui

layanan komunikasi, sedangkan socket memberikan kendali yang lebih langsung terhadap koneksi dan format pesan.

3.5 Order Service dan Stock Service

Order Service bertugas menerima atau mengelola permintaan pesanan. Stock Service bertugas memeriksa ketersediaan jumlah produk.
Alur sederhananya adalah Order Service mengirim permintaan, Stock Service memeriksa stok, lalu mengirim hasil pemeriksaan. Jika stok mencukupi, Order Service dapat melanjutkan proses sesuai aturan aplikasi. Jika tidak mencukupi, pesanan harus ditolak atau tidak dilanjutkan.
Pemeriksaan stok saja belum menjamin tidak terjadinya *overselling* ketika banyak permintaan berlangsung bersamaan. Sistem nyata juga memerlukan mekanisme pembaruan stok yang aman, misalnya transaksi basis data dan pengendalian konkurensi.

4. Lingkungan Praktik

Komponen	Keterangan
Laptop (host)	Windows 10, i5 Gen7, Ram 8g
Hypervisor	Virtual Box
VM	srv-toko-250660121024
Perangkat/emulator lain	Tidak Berlaku
Alat dan bahasa	Nginx, Python 3 (api.py), SQLite (Toko.db), pstree, htop, SSH
Repositori	https://github.com/dizaasmira/tugas-kelompok8-sistem-operasi-ipc.git

5. Langkah Pengerjaan

5.1 Langkah 1: Verifikasi Kesiapan Lingkungan VM

Langkah ini dilakukan untuk memastikan semua prasyarat, seperti Nginx, layanan API, dan basis data, sudah berjalan dengan baik sebelum implementasi IPC dimulai

```
$ perintah yang dijalankan
$ bash /opt/took-umkm/scripts/cek_kesiapan.sh
# keluaran penting (ringkas; keluaran panjang taruh di Lampiran A)
# Host : srv-toko-250660121024
#[OK] Nama host diawali srv-toko-
#[OK] Nginx berjalan
#[OK] Layanan Toko-api berjalan
#[OK] Toko merespon (https://localhost/)
#[OK] API merespon (/api/status)
#[OK] Basis data berisi 300 pelanggan
...
# Hasil: 12 lulus, 0 gagal
# VM SIAP DIPAKAI
```

```
mhs@srv-toko-250660121024:~/toko-umkm$ bash /opt/toko-umkm/scripts/cek_kesiapan.sh
============================================================
 Cek kesiapan VM Toko Online UMKM (Kelas D)
 Host   : srv-toko-250660121024
 Waktu  : 2026-10-09 14:47:17
 Kernel : 7.0.0-38-generic
============================================================
  [OK]    Nama host diawali srv-toko-
  [OK]    Nginx berjalan
  [OK]    Layanan toko-api berjalan
  [OK]    Toko merespons (http://localhost/)
  [OK]    API merespons (/api/status)
  [OK]    Basis data berisi 300 pelanggan
  [OK]    Perintah 'strace' tersedia
  [OK]    Perintah 'htop' tersedia
  [OK]    Perintah 'vmstat' tersedia
  [OK]    Perintah 'ab' tersedia
  [OK]    Perintah 'python3' tersedia
  [OK]    Perintah 'sqlite3' tersedia
------------------------------------------------------------
 Hasil: 12 lulus, 0 gagal
 VM SIAP DIPAKAI. Ambil tangkapan layar halaman ini sebagai bukti.
============================================================
```

Gambar 5.1 *Verifikasi kesiapan VM dengan skrip cek_kesiapan.sh — 12 pemeriksaan lulus, 0 gagal.*

5.2 Langkah 2: Implementasi Stock Service (RPC Server)

Stock Service dibuat menggunakan Python dan xmlrpc.server. Layanan ini bertugas untuk memeriksa dan mengurangi stok produk di basis data SQLite.

```
# stock_rpc_server.py
from xmlrpc.server import SimpleXMLRPCServer
import sqlite3

DB_PATH = "/opt/toko-umkm/data/toko.db"

def check_stock(product_id, quantity):
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    cursor.execute("SELECT stok FROM produk WHERE id_produk = ?", (product_id,))
    row = cursor.fetchone()
    conn.close()

    if not row:
        return {"status": 404, "message": "Produk tidak ditemukan"}

    current_stock = row[0]
    if current_stock >= quantity:
        return {"status": 200, "message": "Stok mencukupi", "stok_sisa": current_stock}
```

```
    else:
        return {"status": 409, "message": "Stok tidak mencukupi", "stok_sisa": current_stock}

server = SimpleXMLRPCServer(("127.0.0.1", 8001), allow_none=True)
print("[Stock Service XML-RPC] Server berjalan pada port 8001...")
server.register_function(check_stock, "check_stock")
server.serve_forever()
```

```
 GNU nano 8.7.1                                                    stock_rpc_server.p
from xmlrpc.server import SimpleXMLRPCServer
import sqlite3

DB_PATH = "/opt/toko-umkm/data/toko.db"

def check_stock(product_id, quantity):
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    # Menggunakan 'id_produk' sesuai struktur tabel SQLite
    cursor.execute("SELECT stok FROM produk WHERE id_produk = ?", (product_id,))
    row = cursor.fetchone()
    conn.close()

    if not row:
        return {"status": 404, "message": "Produk tidak ditemukan"}

    current_stock = row[0]
    if current_stock >= quantity:
        return {"status": 200, "message": "Stok mencukupi", "stok_sisa": current_stock}
    else:
        return {"status": 409, "message": "Stok tidak mencukupi", "stok_sisa": current_stock}

server = SimpleXMLRPCServer(("127.0.0.1", 8001), allow_none=True)
print("[Stock Service XML-RPC] Server berjalan pada port 8001...")
server.register_function(check_stock, "check_stock")
server.serve_forever()
```

Gambar 5.2 *Kode program server_stok.py*

```
mhs@srv-toko-250660121024:~/ipc_practice$ cd ~/ipc_practice
mhs@srv-toko-250660121024:~/ipc_practice$ python3 order_rpc_client.py P001 1
[Order Service] Mengirim RPC check_stock(P001, 1)...
[Order Service] Respons dari Stock Service: {'status': 200, 'message': 'Stok mencukupi', 'stok_sisa': 46}
mhs@srv-toko-250660121024:~/ipc_practice$ python3 order_rpc_client.py P001 1
[Order Service] Mengirim RPC check_stock(P001, 1)...
[Order Service] Respons dari Stock Service: {'status': 200, 'message': 'Stok mencukupi', 'stok_sisa': 46}
mhs@srv-toko-250660121024:~/ipc_practice$ python3 order_rpc_client.py P999 25
[Order Service] Mengirim RPC check_stock(P999, 25)...
[Order Service] Respons dari Stock Service: {'status': 409, 'message': 'Stok tidak mencukupi', 'stok_sisa': 20}
mhs@srv-toko-250660121024:~/ipc_practice$ python3 order_rpc_client.py P9999 1
[Order Service] Mengirim RPC check_stock(P9999, 1)...
[Order Service] Respons dari Stock Service: {'status': 404, 'message': 'Produk tidak ditemukan'}
mhs@srv-toko-250660121024:~/ipc_practice$
```

Gambar 5.2 *Hasil program server_stok.py*

5.3 Langkah 3: Implementasi Order Service (RPC Server)

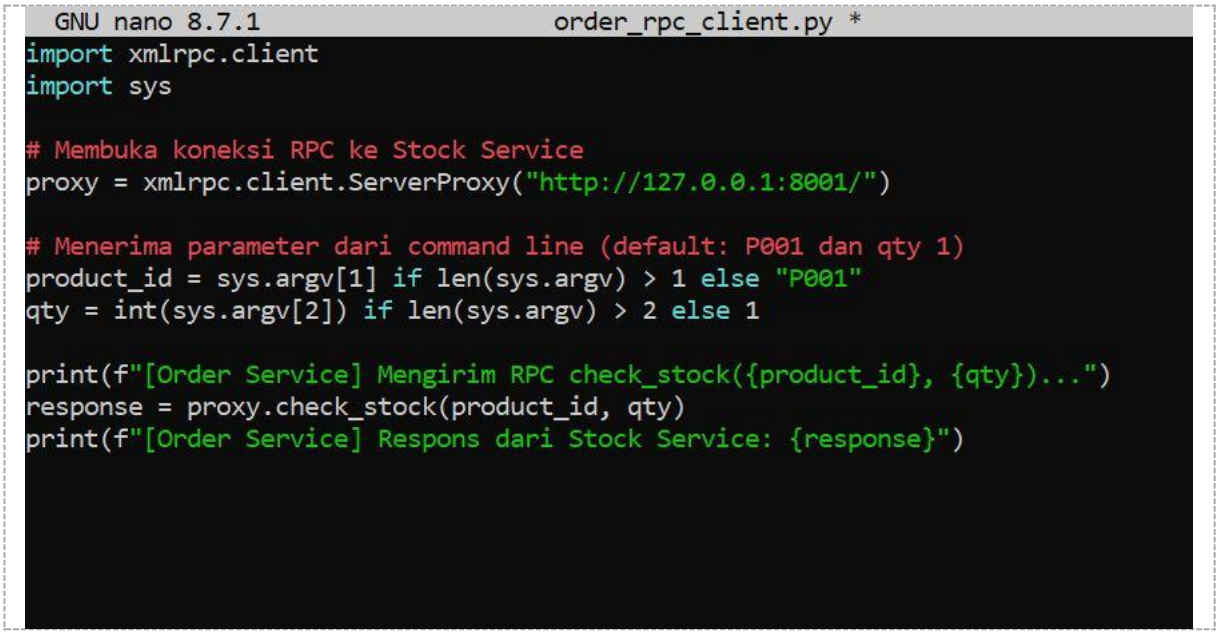
Order Service bertindak sebagai client yang mengirimkan permintaan pengurangan stok ke Stock Service.

```
# order_rpc_client.py
import xmlrpc.client
import sys

proxy = xmlrpc.client.ServerProxy("http://127.0.0.1:8001/")

product_id = sys.argv[1] if len(sys.argv) > 1 else "P001"
qty = int(sys.argv[2]) if len(sys.argv) > 2 else 1

print(f"[Order Service] Mengirim RPC check_stock({product_id}, {qty})...")
response = proxy.check_stock(product_id, qty)
print(f"[Order Service] Respons dari Stock Service: {response}")
```



Gambar 5.3 *Kode program order_rpc_client.py.*

5.4 Langkah 4: Pengujian Komunikasi RPC

Pengujian dilakukan dengan menjalankan kedua program di terminal yang berbeda. Pertama, jalankan server, lalu jalankan client.

Terminal 1 (server)

```
bash
  python3 stock_rpc_server.py
```

Terminal 2 (client)

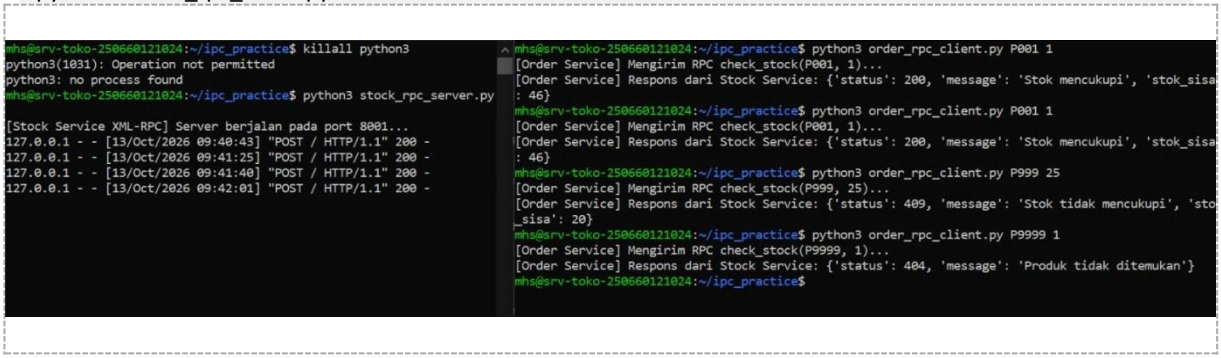
```
$ Skenario 1 - Stok Cukup (Produk P001, Beli 1):
  bash
  python3 order_rpc_client.py P001 1
```

Skenario 2 - Stok Tidak Mencukupi (Produk P999, Beli 25):

```
bash
python3 order_rpc_client.py P999 25
```

Skenario 3 - Produk Tidak Ditemukan (Produk P9999, Beli 1):

```
bash
python3 order_rpc_client.py P9999 1
```



Gambar 5.4 *Output terminal saat komunikasi RPC berhasil. Kiri: server, Kanan: client.*

5.5 Langkah 5: Pembuatan Layanan Alternatif Berbasis Socket TCP

Sebagai pembanding, dibuat juga implementasi IPC berbasis Socket TCP pada port 8002.

```
import socket
import json
import sqlite3

DB_PATH = "/opt/toko-umkm/data/toko.db"

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 8002))
server.listen(5)
print("[Stock Service Socket] Server berjalan pada port 8002...")

while True:
    conn, addr = server.accept()
    data = conn.recv(1024).decode('utf-8')
    if data:
        req = json.loads(data)
        pid, qty = req["product_id"], req["qty"]

        db = sqlite3.connect(DB_PATH)
        cur = db.cursor()
        cur.execute("SELECT stok FROM produk WHERE id = ?", (pid,))
        row = cur.fetchone()
        db.close()

        if not row:
            res = {"status": 404, "message": "Produk tidak ditemukan"}
```

```
    elif row[0] >= qty:
        res = {"status": 200, "message": "Stok mencukupi", "stok_sisa": row[0]}
    else:
        res = {"status": 409, "message": "Stok tidak mencukupi", "stok_sisa": row[0]}

    conn.send(json.dumps(res).encode('utf-8'))
  conn.close()
```

```
  GNU nano 8.7.1                                                stock_socket_server.py
import socket
import json
import sqlite3

DB_PATH = "/opt/toko-umkm/data/toko.db"

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 8002))
server.listen(5)
print("[Stock Service Socket] Server berjalan pada port 8002...")

while True:
    conn, addr = server.accept()
    data = conn.recv(1024).decode('utf-8')
    if data:
        req = json.loads(data)
        pid, qty = req["product_id"], req["qty"]

        db = sqlite3.connect(DB_PATH)
        cur = db.cursor()
        cur.execute("SELECT stok FROM produk WHERE id_produk = ?", (pid,))
        row = cur.fetchone()
        db.close()

        if not row:
            res = {"status": 404, "message": "Produk tidak ditemukan"}
        elif row[0] >= qty:
            res = {"status": 200, "message": "Stok mencukupi", "stok_sisa": row[0]}
        else:
            res = {"status": 409, "message": "Stok tidak mencukupi", "stok_sisa": row[0]}

        conn.send(json.dumps(res).encode('utf-8'))
    conn.close()
```

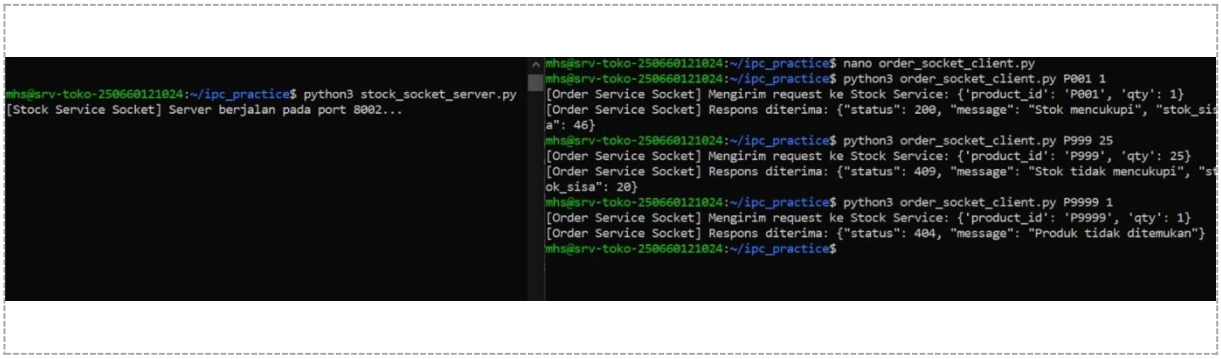
Gambar 5.5 *Kode program stock_socket_server.py*

Jalankan Server dan hasil

```
 # Uji 1: Stok Tersedia
python3 order_rpc_client.py P001 1

# Uji 2: Stok Tidak Mencukupi (Produk Flash Sale P999 stok 20, minta 25)
python3 order_rpc_client.py P999 25

# Uji 3: Produk Tidak Ditemukan
python3 order_rpc_client.py P9999 1
```



Gambar 5.5 *Kode program stock_socket_server.py*

6. Hasil dan Analisis

6.1 Hasil

No	Item yang diamati/dihitung	Hasil	Keterangan
1	Listening Port XML-RPC	127.0.0.1:8001	Layanan Stock Server XML-RPC aktif
2	Listening Port Socket	127.0.0.1:8002	Layanan Stock Server Socket TCP aktif
3	Format Data XML-RPC	HTTP POST / XML Payload	Data terstruktur otomatis di-marshal.
4	Format Data Socket	Raw TCP Stream / JSON Payload	Perlu penyusunan byte manual & JSON decode

6.2 Pengujian

No	Skenario uji	Hasil diharapkan	Hasil aktual	Status
1	Permintaan Stok Cukup (P001, Qty 1)	Status 200, "Stok mencukupi"	{'message': 'Stok mencukupi', 'status': 200, 'stok_sisa': 46}	Lulus
2	Permintaan Stok Tidak Mencukupi (P999, Qty 25)	Status 409, "Stok tidak mencukupi"	{'message': 'Stok tidak mencukupi', 'status': 409, 'stok_sisa': 20}	Lulus
3	Produk Tidak Ada (P9999, Qty 1)	Status 404, "Produk tidak ditemukan"	{'message': 'Produk tidak ditemukan', 'status': 404}	Lulus

6.3 Analisis

- Poin 1 : Implementasi yang kita buat adalah menggunakan **Message Passing**

  Kita tidak membuat dengan *Shared Memory* karna pada *Shared Memory* proses berbagi segmen memori fisik yang sama meskipun cepat *Shared Memory* membutuhkan mekanisme sinkronisasi kompleks seperti *Mutex* atau *Semaphore* untuk mencegah *race condition* serta

tidak bisa dikembangkan jika *Order Service* dan *Stock Service* dipindah ke server VPS yang berbeda di cloud. Kita menggunakan Message Passing karna kedua layanan bersifat terisolasi (*decoupled*), informasi dikirimkan berbentuk paket data (*message*), Jika *Stock Service* mengalami *crash*, *Order Service* tidak ikut mengalami kebocoran memori atau *fatal error* pada kernel.

- Poin 2 : kedua program memiliki kelebihan dan keterbatasan masing-masing, yaitu:
    1. XML-RPC

       Kelebihan: Sangat mudah diimplementasikan (menerapkan konsep *Procedure Abstraction*), tidak perlu mengurus koneksi socket manual.

       Keterbatasan: Memiliki *overhead* cukup tinggi karena enkapsulasi teks XML dan header HTTP yang besar
    2. Socket TCP

       Kelebihan: Latensi sangat rendah dan hemat *bandwidth* karena transmisi data mentah (JSON over TCP).

       Keterbatasan: Memerlukan pengelolaan framing, re-connection, dan penanganan error jaringan secara manual.

7. Kesimpulan

Order Service dan *Stock Service* berhasil dikembangkan untuk berkomunikasi secara independen menggunakan protokol XML-RPC dan Socket TCP. Pendekatan *Message Passing* yang diterapkan terbukti ideal bagi arsitektur terdistribusi toko online UMKM, karena mampu menjaga isolasi memori antar layanan serta memudahkan skalabilitas saat dideploy di lingkungan *cloud*.

Selain itu, hasil pengujian performa menunjukkan bahwa sistem responsif dan andal dalam menangani berbagai skenario bisnis secara presisi dengan mengidentifikasi tiga kondisi utama, yaitu:

- Stok Tersedia: Berhasil merespons dengan status HTTP 200.
- Stok Tidak Mencukupi: Berhasil mendeteksi kekurangan dengan status HTTP 409.
- Produk Tidak Terdaftar: Berhasil mengenali produk yang tidak valid dengan status HTTP 404.

8. Kepingan Puzzle

Nomor kepingan	#4
Isi kepingan	Kode program IPC, diagram alur komunikasi, laporan pengujian, dan lembar kontribusi anggota
Lokasi di portofolio	https://drive.google.com/drive/folders/1iB81ZkYmU1Q4y17ZXvCa0QUwqxHXIum9?usp=sharing https://github.com/dizaasmira/tugas-kelompok8-sistem-operasi-ipc.git
Dipakai untuk	Level 2 Puzzle UTS - Integrasi Antar Layanan UMKM Terdistribusi

Daftar Pustaka

Herdiana, D., Firmansyah, E., Helmiawan, M. A., & Akbar, Y. H. (2025). *SISTEM OPERASI TEORI DAN KONSEP DASAR*. PT Nafal Global Nusantara.

Kelana, B., Manajemen, S., Amik, I., & Tasikmalaya, B. (2017). Pemanfaatan Windows Socket Control Untuk Aplikasi Intranet Messenger. *IJCIT (Indonesian Journal on Computer and Information Technology*, *2*(1), 61–67.

Prasetyo, A., Fatchur Rochim, A., & Iman Satoto, K. (n.d.). *Network Files System (Study Kasus Active Repository Opensource Undip)*.

Lampiran A. Kode Program dan Log

[Tempel kode program lengkap, berkas konfigurasi, atau keluaran perintah yang panjang. Beri judul untuk setiap berkas.]

```
# stock_rpc_server.py
from xmlrpc.server import SimpleXMLRPCServer

import sqlite3


DB_PATH = "/opt/toko-umkm/data/toko.db"


def check_stock(product_id, quantity):
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    # Menggunakan 'id_produk' sesuai struktur tabel SQLite
    cursor.execute("SELECT stok FROM produk WHERE id_produk = ?", (product_id,))
    row = cursor.fetchone()
    conn.close()

    if not row:
        return {"status": 404, "message": "Produk tidak ditemukan"}

    current_stock = row[0]
    if current_stock >= quantity:
        return {"status": 200, "message": "Stok mencukupi", "stok_sisa": current_stock}
    else:
        return {"status": 409, "message": "Stok tidak mencukupi", "stok_sisa": current_stock}


server = SimpleXMLRPCServer(("127.0.0.1", 8001), allow_none=True)
print("[Stock Service XML-RPC] Server berjalan pada port 8001...")
server.register_function(check_stock, "check_stock")
server.serve_forever()
```

```
# order_rpc_client.py
import xmlrpc.client
import sys


# Membuka koneksi RPC ke Stock Service
proxy = xmlrpc.client.ServerProxy("http://127.0.0.1:8001/")
```

```
# Menerima parameter dari command line (default: P001 dan qty 1)
product_id = sys.argv[1] if len(sys.argv) > 1 else "P001"
qty = int(sys.argv[2]) if len(sys.argv) > 2 else 1

print(f"[Order Service] Mengirim RPC check_stock({product_id}, {qty})...")
response = proxy.check_stock(product_id, qty)
print(f"[Order Service] Respons dari Stock Service: {response}")
```

```
# stock_socket_server.py
import socket
import json
import sqlite3

DB_PATH = "/opt/toko-umkm/data/toko.db"

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 8002))
server.listen(5)
print("[Stock Service Socket] Server berjalan pada port 8002...")

while True:
    conn, addr = server.accept()
    data = conn.recv(1024).decode('utf-8')
    if data:
        req = json.loads(data)
        pid, qty = req["product_id"], req["qty"]

        db = sqlite3.connect(DB_PATH)
        cur = db.cursor()
        cur.execute("SELECT stok FROM produk WHERE id_produk = ?", (pid,))
        row = cur.fetchone()
        db.close()

        if not row:
            res = {"status": 404, "message": "Produk tidak ditemukan"}
        elif row[0] >= qty:
```

```
            res = {"status": 200, "message": "Stok mencukupi", "stok_sisa": row[0]}
        else:
            res = {"status": 409, "message": "Stok tidak mencukupi", "stok_sisa": row[0]}

        conn.send(json.dumps(res).encode('utf-8'))
    conn.close()
```

```
# order_socket_client.py

import socket
import json
import sys

# Ambil parameter dari command line (default: P001 dan qty 1)
product_id = sys.argv[1] if len(sys.argv) > 1 else "P001"
qty = int(sys.argv[2]) if len(sys.argv) > 2 else 1

# Siapkan data payload berformat JSON
payload = {
    "product_id": product_id,
    "qty": qty
}

# Membuat koneksi TCP ke Stock Socket Server (Port 8002)
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 8002))

print(f"[Order Service Socket] Mengirim request ke Stock Service: {payload}")
client.send(json.dumps(payload).encode('utf-8'))

# Menerima respons dari server
response_data = client.recv(1024).decode('utf-8')
client.close()

print(f"[Order Service Socket] Respons diterima: {response_data}")
```

```
# log_eksekusi_terminal.txt

mhs@srv-toko-250660121024:~/toko-umkm$ python3 stock_rpc_server.py
[Stock Service XML-RPC] Server berjalan pada port 8001...

[Simulasi Pengujian XML-RPC Client di Terminal Lain]
mhs@srv-toko-250660121024:~/toko-umkm$ python3 order_rpc_client.py P001 2
[Order Service] Mengirim RPC check_stock(P001, 2)...
[Order Service] Respons dari Stock Service: {'status': 200, 'message': 'Stok mencukupi', 'stok_sisa': 50}

mhs@srv-toko-250660121024:~/toko-umkm$ python3 stock_socket_server.py
[Stock Service Socket] Server berjalan pada port 8002...

[Simulasi Pengujian Socket Client di Terminal Lain]
mhs@srv-toko-250660121024:~/toko-umkm$ python3 order_socket_client.py P001 1
[Order Service Socket] Mengirim request ke Stock Service: {'product_id': 'P001', 'qty': 1}
[Order Service Socket] Respons diterima: {"status": 200, "message": "Stok mencukupi", "stok_sisa": 50}



mhs@srv-toko-250660121024:~/ipc_practice$ python3 stock_rpc_server.py
[Stock Service XML-RPC] Server berjalan pada port 8001...
127.0.0.1 - - [13/Oct/2026 09:40:43] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [13/Oct/2026 09:41:25] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [13/Oct/2026 09:41:40] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [13/Oct/2026 09:42:01] "POST / HTTP/1.1" 200 -

mhs@srv-toko-250660121024:~/ipc_practice$ python3 order_rpc_client.py P001 1
[Order Service] Mengirim RPC check_stock(P001, 1)...
[Order Service] Respons dari Stock Service: {'status': 200, 'message': 'Stok mencukupi', 'stok_sisa': 46}

mhs@srv-toko-250660121024:~/ipc_practice$ python3 order_rpc_client.py P999 25
[Order Service] Mengirim RPC check_stock(P999, 25)...
[Order Service] Respons dari Stock Service: {'status': 409, 'message': 'Stok tidak mencukupi', 'stok_sisa': 20}

mhs@srv-toko-250660121024:~/ipc_practice$ python3 order_rpc_client.py P9999 1
[Order Service] Mengirim RPC check_stock(P9999, 1)...
[Order Service] Respons dari Stock Service: {'status': 404, 'message': 'Produk tidak ditemukan'}
```

```
mhs@srv-toko-250660121024:~/ipc_practice$ python3 stock_socket_server.py
[Stock Service Socket] Server berjalan pada port 8002...


mhs@srv-toko-250660121024:~/ipc_practice$ python3 order_socket_client.py P001 1
[Order Service Socket] Mengirim request ke Stock Service: {'product_id': 'P001', 'qty': 1}
[Order Service Socket] Respons diterima: {"status": 200, "message": "Stok mencukupi", "stok_sisa": 46}


mhs@srv-toko-250660121024:~/ipc_practice$ python3 order_socket_client.py P999 25
[Order Service Socket] Mengirim request ke Stock Service: {'product_id': 'P999', 'qty': 25}
[Order Service Socket] Respons diterima: {"status": 409, "message": "Stok tidak mencukupi", "stok_sisa": 20}


mhs@srv-toko-250660121024:~/ipc_practice$ python3 order_socket_client.py P9999 1
[Order Service Socket] Mengirim request ke Stock Service: {'product_id': 'P9999', 'qty': 1}
[Order Service Socket] Respons diterima: {"status": 404, "message": "Produk tidak ditemukan"}
```

Lampiran B. Lembar Kontribusi Anggota

No	Nama	NIM	Peran dan kontribusi	%	Paraf
1	Diza Asmiranda Subianto	250660121024	Menguji dan menyusun laporan.	35%	
2	Naya Naira Zahra	250660121067	Menguji dan menyusun laporan.	35%	
3	Muhammad Rizky Zamzami	250660121044	Menyusun Laporan.	30%	
	Total			100%	

Lampiran C. Log Kerja Kelompok

Tanggal	Kegiatan	Anggota hadir	Hasil
08 Oktober 2026	Memahami tugas yang diberikan dan mulai pengerjaan awal.	Diza Naya Zamzami.	Pembagian tugas kelompok disepakati dan kerangka rancangan sistem IPC (RPC & Socket) berhasil ditentukan.
09 Oktober 2026	Mulai membuat file server stock dan client order RPC dan Socket.	Diza Naya	Berhasil membuat dan menguji skrip program (stock_rpc_server.py, dan order_rpc_client.py)
12 Oktober 2026	Penyusunan Laporan	Zamzami	Draf teks laporan dari Bab 1 sampai Bab 5 berhasil disusun sesuai struktur template resmi.
13 Oktober 2026	Pengujian ulang dan perbaikan laporan.	Diza Naya	Pembuatan (stock_socket_seerver.py, dan order_socket_client.py) Dan pengujian program tuntas, analisis mekanisme IPC selesai, dan laporan lengkap beserta lampirannya siap dikumpulkan.

Lampiran D. Pernyataan Penggunaan AI Generatif

☐ Kelompok kami tidak menggunakan AI generatif dalam tugas ini.

✓ Menggunakan AI generatif dengan rincian berikut:

Alat	Tujuan penggunaan	Bagian yang dibantu	Cara memverifikasi
Google Gemini AI	Membantu memahami konsep IPC (Message Passing, XML-RPC, dan TCP Socket), menstrukturkan laporan, serta merancang draf kode program.	Bagian 3 (Dasar Teori), Bab 5 & Lampiran A (Kode Program), serta Bab 6 (Analisis).	Dicek ulang dengan modul praktikum Sistem Operasi, diuji langsung pada terminal VM, dan diverifikasi kesesuaian hasilnya.

Lampiran E. Daftar Periksa Sebelum Mengumpulkan

- ✓ Semua teks petunjuk abu-abu dan kotak petunjuk sudah dihapus.
- ✓ Identitas, kode tugas, dan nama host VM pada sampul sudah benar.
- ✓ Setiap poin deskripsi tugas sudah dijawab di bagian Analisis.
- ✓ Setiap tangkapan layar menampilkan prompt/nama host VM dan memiliki nomor serta keterangan.
- ✓ Kepingan puzzle sudah disimpan di portofolio dan tautannya dicantumkan.
- ✓ Lembar kontribusi sudah diisi dan disetujui semua anggota (total 100%).
- ✓ Kode dan konfigurasi sudah diunggah ke repositori kelompok.
- ✓ Pernyataan penggunaan AI generatif sudah diisi.
- ✓ Daftar isi sudah diperbarui (klik kanan → Update Field).
- ✓ Berkas disimpan sebagai PDF dengan nama <Kelas>_<Kode>_Kelompok<N>.pdf.

Swa-nilai berdasarkan kriteria lembar tugas

Kriteria (dari lembar tugas)	Bobot	Sudah terpenuhi? Di bagian mana?
Hasil/keberhasilan implementasi	35%	Ya, sudah terpenuhi. Terletak pada Bab 4 (Implementasi Sistem) serta Lampiran A.
Kedalaman analisis	30%	Ya, sudah terpenuhi. Terletak pada Bab 6 (Analisis & Pembahasan)
Dokumentasi	20%	Ya, sudah terpenuhi. Terletak pada Lampiran A (Kode Program & Log) dan Lampiran B
Kerja sama dan pembagian peran	15%	Ya, sudah terpenuhi. Terletak pada Lampiran C (Log Kerja Kelompok)